ASIAN SCHOOL OF BUSINESS

Programme	**:**	**BCA (Honours) AI and Data Science**
Course Name	**:**	**Database Management Systems**
Course Code	**:**	25-9222-304
Given Date	**:**	1-10-26
Date of Submission	**:**	12-10-26

Assignment - 5

Topic: Payment Management Systems

Submitted to,

Athira A G

Assistant Professor

Submitted by,

Name: Shivganga R T

Class: BCA

SKILL DEVELOPMENT WEEK 5
PAYMENT MANAGEMENT SYSTEM

INTRODUCTION

The **Payment Management System** is a database project designed to store and manage customer payment transactions using **MySQL**. Its objectives are to create **Payment** table, record critical payment details such as payment mode, date, amount, and status, separate and manage successful and failed transactions, analyze payment methods used by customers, and generate simple payment transaction reports.

2. DATABASE DESIGN

2.1 Payment Table

The Payment table stores transactional metrics including payment methods, dates, amounts, and statuses.

Attribute	Data Type	Description
payment_id	INT	Primary key
customer_id	INT	Customer identifier
amount	DECIMAL(10,2)	Payment amount
payment_mode	VARCHAR(30)	Payment method (e.g., UPI, Card)
payment_date	DATE	Date of payment
payment_status	VARCHAR(20)	Success or Failed
transaction_reference	VARCHAR(100)	Transaction reference number

3. SQL IMPLEMENTATION

3.1 Create the Database

```
CREATE DATABASE PaymentDB;
USE PaymentDB;
```

3.2 Create the Payment Table

- Create Payment Table

  CREATE TABLE Payment (payment_id INT PRIMARY KEY, customer_id INT, amount DECIMAL(10,2), payment_mode VARCHAR(30), payment_date DATE, payment_status VARCHAR(20), transaction_reference VARCHAR(100));

4. INSERT SAMPLE PAYMENT RECORDS

4.1 Insert realistic payment logs into the table

```
INSERT INTO Payment VALUES
(1, 101, 1500.00, 'UPI', '2026-10-10', 'Success', 'TXN1001'),
(2, 102, 2500.00, 'Card', '2026-10-11', 'Failed', 'TXN1002'),
(3, 103, 800.00, 'Cash', '2026-10-11', 'Success', 'TXN1003'),
(4, 104, 3200.00, 'Net Banking', '2026-10-12', 'Failed', 'TXN1004'),
(5, 105, 1200.00, 'UPI', '2026-10-12', 'Success', 'TXN1005');
```

5. MANAGE SUCCESSFUL AND FAILED TRANSACTIONS

5.1 View Successful Payments

Filters the database log to reveal completely fulfilled transactions.

```
SELECT * FROM Payment WHERE payment_status = 'Success';
```

5.2 View Failed Payments

Isolates problematic logs that require intervention or alternative checkout methods.

```
SELECT * FROM Payment WHERE payment_status = 'Failed';
```

5.3 Update a Failed Transaction

Modifies a failed flag into a successful record after external reconciliation matches values.

```
UPDATE Payment SET payment_status = 'Success' WHERE payment_id = 2;
```

6. ANALYZE PAYMENT METHODS USED BY CUSTOMERS

6.1 Count Transactions by Payment Method

Counts general transactional volume across each chosen option channel.

```
SELECT payment_mode, COUNT(*) AS total_transactions FROM Payment GROUP BY payment_mode ORDER BY total_transactions DESC;
```

6.2 Analyze Successful Payments by Method

Aggregates summary totals to find out which transaction options generate the most revenue.

```
SELECT payment_mode, COUNT(*) AS successful_transactions, SUM(amount) AS total_amount FROM Payment WHERE payment_status = 'Success' GROUP BY payment_mode;
```

7. GENERATE PAYMENT TRANSACTION REPORTS

7.1 Complete Transaction Report

Generates a raw historical transaction breakdown sorted chronologically.

```
SELECT payment_id, customer_id, amount, payment_mode, payment_date, payment_status FROM Payment ORDER BY payment_date DESC;
```

7.2 Payment Status Summary

Provides a direct financial perspective of total successful deposits vs total failures.

```
SELECT payment_status, COUNT(*) AS transaction_count, SUM(amount) AS total_amount FROM Payment GROUP BY payment_status;
```

CONCLUSION

The **Payment Management System** provides a clean, entry-level approach to storing and analyzing financial records using **MySQL**. By discarding unnecessary database constraints like auto-generation scripts, structural check constraints, or automated timestamps, the code runs transparently while fully capturing all critical processing details. Implementing straightforward data filters, updating failed elements, and running simple group functions offers standard dashboard metrics for tracking customer checkout habits and general store liquidity.